Name	Vedant Dinkar Patil
UID no.	2024300170
Experiment No.	7

AIM:	Implement Auto Associative and Bidirectional Associative Memory Network
PROBLEM STATEMENT :	1.Train the autoassociative network for input vector [-111 1] and also test the network for the same input vector. Test the autoassociative network with one missing, one mistake, two missing and two mistake entries in test vector. 2. Train a heteroassociative memory network to store the input vectors s = (s,52,53,5) to the output vectors t =(t_1,t2). The vector pairs are given in Table 3. Also test the performance of the network using its training input as testing input.
Theory:	**Overview of Associative Memory Networks** Associative memory networks are specialized neural network models designed to store specific patterns and recall them later when presented with similar, partial, or corrupted versions. Instead of relying on explicit memory addresses, these networks operate on content addressability. **1. Autoassociative Memory** • **Definition:** An autoassociative memory network stores a set of patterns and is trained to recall the exact original pattern when given a noisy, incomplete, or corrupted version of that same input. • **How it Works:** The network maps an input vector to itself. During training, a weight matrix W is generated from the training vectors. During testing, the network processes a test input vector through the weight matrix and applies activation functions to successfully reconstruct the original stored pattern, provided the input does not have too many missing or mistaken entries. **2. Heteroassociative Memory** • **Definition:** Unlike autoassociative networks, a heteroassociative memory network associates a given input pattern (s) with a completely *different* target output pattern (t). • **Architecture:** The network consists of an input layer connected to an output layer via directional weights. When an input vector is applied, the network

	computes the net input and maps it to the desired output vector. **3. Bidirectional Associative Memory (BAM)** • **Definition:** BAM is a heteroassociative memory network that allows information flow in two directions. It can recall an output pattern given an input pattern, or reverse the process to recall the input pattern given an output pattern. • **Practical Application:** BAM is frequently used for pattern recognition and classification problems, such as associating grid-based visual characters (like letters 'E' and 'F') with specific bipolar target responses. **4. Core Mathematical Principles** • **Outer Product Rule:** The weights of the network are typically computed by summing the outer products of the training input and target vectors ($W = \sum s^T t$). For autoassociative networks, since the input maps to itself, the weight matrix is derived from the outer product of the input vector with itself ($W = \sum x^T x$). • **Net Input Calculation:** To test the network, the input vector x is multiplied by the total weight matrix W to find the net input ($y_{in} = x \cdot W$). • **Activation Functions:** Activation functions (such as sign or threshold functions) are subsequently applied to the net input to yield clean, discrete binary or bipolar output responses.

PROGRAM:	ASSOCIATIVE :
	#include <iostream>
	#include <vector>
	
	using namespace std;
	
	int main() {
	// The original input vector to train the network
	vector<int> original_x = {-1, 1, 1, 1};
	int n = original_x.size();
	
	// Initialize weight matrix W
	vector<vector<int>> W(n, vector<int>(n));
	
	// Train the network using outer product rule: W = x^T * x
	for (int i = 0; i < n; i++) {
	for (int j = 0; j < n; j++) {
	W[i][j] = original_x[i] * original_x[j];

```
    }
}

int choice = 1;
while (choice == 1) {
    vector<int> x(n);
    cout << "\n-------------------------------------------------\n";
    cout << "Enter the 4 elements of the test vector (space-separated): ";
    for (int i = 0; i < n; i++) {
        cin >> x[i];
    }

    // 1. Show the Weight Matrix W first
    cout << "\nWeight Matrix W:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << W[i][j] << "\t";
        }
        cout << "\n";
    }

    // 2. Compute net input y_in = x * W
    vector<int> y_in(n, 0);
    for (int j = 0; j < n; j++) {
        for (int i = 0; i < n; i++) {
            y_in[j] += x[i] * W[i][j];
        }
    }

    // Show y_in
    cout << "\nNet input y_inj = [ ";
    for (int j = 0; j < n; j++) {
        cout << y_in[j] << " ";
    }
    cout << "]\n";

    // 3. Apply activation function
    vector<int> y(n);
    for (int j = 0; j < n; j++) {
        if (y_in[j] > 0) {
```

```
            y[j] = 1;
        } else if (y_in[j] < 0) {
            y[j] = -1;
        } else {
            y[j] = 0; // Output is 0 when net input is 0
        }
    }

    // Show final output after activation
    cout << "Output after activation y = [ ";
    for (int j = 0; j < n; j++) {
        cout << y[j] << " ";
    }
    cout << "]\n\n";

    // 4. Check if it gives the correct answer
    bool correct = true;
    for (int j = 0; j < n; j++) {
        if (y[j] != original_x[j]) {
            correct = false;
            break;
        }
    }

    if (correct) {
        cout << "Result: Correct response is obtained.\n";
    } else {
        cout << "Result: Incorrect response. The network did not recognize the input.\n";
    }

    cout << "\nDo you want to test another input? (1 for Yes, 0 for No): ";
    cin >> choice;
  }

  return 0;
}
```

HETEROASSOCIATIVE:

```
#include <iostream>
#include <vector>
```

```
using namespace std;

int main() {
    // Training data from Table 3 (4 input vectors, 4 target vectors)
    vector<vector<int>> S = {
        {1, 0, 0, 0},
        {1, 1, 0, 0},
        {0, 0, 0, 1},
        {0, 0, 1, 1}
    };

    vector<vector<int>> T = {
        {0, 1},
        {0, 1},
        {1, 0},
        {1, 0}
    };

    int num_patterns = 4;
    int input_size = 4;
    int output_size = 2;

    // Initialize weight matrix W (4 rows, 2 columns)
    vector<vector<int>> W(input_size, vector<int>(output_size, 0));

    // Train the network using outer product rule: W = sum(S^T * T)
    for (int p = 0; p < num_patterns; p++) {
        for (int i = 0; i < input_size; i++) {
            for (int j = 0; j < output_size; j++) {
                W[i][j] += S[p][i] * T[p][j];
            }
        }
    }

    int choice = 1;
    while (choice == 1) {
        vector<int> x(input_size);
        cout << "\n-------------------------------------------------\n";
        cout << "Enter the 4 elements of the test vector (e.g., 1 1 0 0): ";
        for (int i = 0; i < input_size; i++) {
```

```
    cin >> x[i];
}

// 1. Show the Target Matrix T first
cout << "\nTarget Matrix T (Training Targets):\n";
for (int p = 0; p < num_patterns; p++) {
    for (int j = 0; j < output_size; j++) {
        cout << T[p][j] << "\t";
    }
    cout << "\n";
}

// 2. Show the Weight Matrix W
cout << "\nTrained Weight Matrix W:\n";
for (int i = 0; i < input_size; i++) {
    for (int j = 0; j < output_size; j++) {
        cout << W[i][j] << "\t";
    }
    cout << "\n";
}

// 3. Compute net input y_in = x * W
vector<int> y_in(output_size, 0);
for (int j = 0; j < output_size; j++) {
    for (int i = 0; i < input_size; i++) {
        y_in[j] += x[i] * W[i][j];
    }
}

// Show net input y_inj
cout << "\nNet input y_inj = [ ";
for (int j = 0; j < output_size; j++) {
    cout << y_in[j] << " ";
}
cout << "]\n";

// 4. Apply activation function f(y_in) and show final output
vector<int> y(output_size);
for (int j = 0; j < output_size; j++) {
    if (y_in[j] > 0) {
```

```
            y[j] = 1;
        } else {
            y[j] = 0;
        }
    }

    cout << "Output after activation y = [ ";
    for (int j = 0; j < output_size; j++) {
        cout << y[j] << " ";
    }
    cout << "]\n\n";

    cout << "Do you want to test another input? (1 for Yes, 0 for No): ";
    cin >> choice;
  }

  return 0;
}
```

BAM:

```
#include <iostream>
#include <vector>

using namespace std;

int activate(int net_input) {
  if (net_input > 0) return 1;
  if (net_input < 0) return -1;
  return 0;
}

int main() {
  vector<vector<int>> X = {
    {1, 1, 1, 1, -1, -1, 1, 1, 1, 1, -1, -1, 1, 1, 1},       // Pattern 'E'
    {1, 1, 1, 1, -1, -1, 1, 1, -1, 1, -1, -1, 1, -1, -1}     // Pattern 'F'
  };

  vector<vector<int>> Y = {
    {-1, 1}, // Target for 'E'
    {1, 1}   // Target for 'F'
  };
```

```
    int num_patterns = 2;
    int input_size = 15;
    int output_size = 2;

    vector<vector<int>> W(input_size, vector<int>(output_size, 0));

    for (int p = 0; p < num_patterns; p++) {
        for (int i = 0; i < input_size; i++) {
            for (int j = 0; j < output_size; j++) {
                W[i][j] += X[p][i] * Y[p][j];
            }
        }
    }

    int choice = 0;
    while (true) {
        cout << "\n================ BIDIRECTIONAL ASSOCIATIVE MEMORY
(BAM) ================\n";
        cout << "1. Insert Input (15 elements) to get Output (Forward Recall)\n";
        cout << "2. Insert Output (2 elements) to get Input (Backward Recall)\n";
        cout << "3. Exit\n";
        cout << "Select an option: ";
        cin >> choice;

        if (choice == 1) {
            vector<int> x_test(input_size);
            cout << "\nEnter 15 elements for the input vector (space-separated bipolar -1 or
1):\n> ";
            for (int i = 0; i < input_size; i++) {
                cin >> x_test[i];
            }

            vector<int> y_in(output_size, 0);
            for (int j = 0; j < output_size; j++) {
                for (int i = 0; i < input_size; i++) {
                    y_in[j] += x_test[i] * W[i][j];
                }
            }
```

```
            cout << "Net Output y_inj = [ ";
            for (int j = 0; j < output_size; j++) cout << y_in[j] << " ";
            cout << "]\n";

            cout << "Final Activated Output = [ ";
            for (int j = 0; j < output_size; j++) cout << activate(y_in[j]) << " ";
            cout << "]\n";

        } else if (choice == 2) {
            vector<int> y_test(output_size);
            cout << "\nEnter 2 elements for the output vector (space-separated bipolar -1 or
1):\n> ";
            for (int j = 0; j < output_size; j++) {
                cin >> y_test[j];
            }

            vector<int> x_in(input_size, 0);
            for (int i = 0; i < input_size; i++) {
                for (int j = 0; j < output_size; j++) {
                    x_in[i] += y_test[j] * W[i][j];
                }
            }

            cout << "Net Input x_inj = [ ";
            for (int i = 0; i < input_size; i++) cout << x_in[i] << " ";
            cout << "]\n";

            cout << "Final Activated Input = [ ";
            for (int i = 0; i < input_size; i++) cout << activate(x_in[i]) << " ";
            cout << "]\n";

        } else if (choice == 3) {
            cout << "Exiting...\n";
            break;
        } else {
            cout << "Invalid choice. Please try again.\n";
        }
    }

    return 0;
```

	}

Output:

```
PS C:\Users\prajw\OneDrive\Desktop> g++ associative.cpp -o associative
PS C:\Users\prajw\OneDrive\Desktop> .\associative

-------------------------------------------------
Enter the 4 elements of the test vector (space-separated): -1 1 1 1

Weight Matrix W:
1       -1      -1      -1
-1      1       1       1
-1      1       1       1
-1      1       1       1

Net input y_inj = [ -4 4 4 4 ]
Output after activation y = [ -1 1 1 1 ]

Result: Correct response is obtained.

Do you want to test another input? (1 for Yes, 0 for No): 1

-------------------------------------------------
Enter the 4 elements of the test vector (space-separated): 0 1 1 1

Weight Matrix W:
1       -1      -1      -1
-1      1       1       1
-1      1       1       1
-1      1       1       1

Net input y_inj = [ -3 3 3 3 ]
Output after activation y = [ -1 1 1 1 ]

Result: Correct response is obtained.

Do you want to test another input? (1 for Yes, 0 for No): 1

-------------------------------------------------
Enter the 4 elements of the test vector (space-separated): -1 -1 1 1

Weight Matrix W:
1       -1      -1      -1
-1      1       1       1
-1      1       1       1
-1      1       1       1
```

```
PS C:\Users\prajw\OneDrive\Desktop> g++ heteroassociative.cpp -o hetero
PS C:\Users\prajw\OneDrive\Desktop> .\hetero

--------------------------------------------------
Enter the 4 elements of the test vector (e.g., 1 1 0 0): 1 0 0 0

Target Matrix T (Training Targets):
0       1
0       1
1       0
1       0

Trained Weight Matrix W:
0       2
0       1
1       0
2       0

Net input y_inj = [ 0 2 ]
Output after activation y = [ 0 1 ]

Do you want to test another input? (1 for Yes, 0 for No): 1

--------------------------------------------------
Enter the 4 elements of the test vector (e.g., 1 1 0 0): 1 1 0 0

Target Matrix T (Training Targets):
0       1
0       1
1       0
1       0

Trained Weight Matrix W:
0       2
0       1
1       0
2       0

Net input y_inj = [ 0 3 ]
Output after activation y = [ 0 1 ]

Do you want to test another input? (1 for Yes, 0 for No): 0
PS C:\Users\prajw\OneDrive\Desktop>
```

```
Net input y_inj = [ -2 2 2 2 ]
Output after activation y = [ -1 1 1 1 ]

Result: Correct response is obtained.

Do you want to test another input? (1 for Yes, 0 for No): 1

--------------------------------------------------
Enter the 4 elements of the test vector (space-separated): 0 0 1 1

Weight Matrix W:
1       -1      -1      -1
-1      1       1       1
-1      1       1       1
-1      1       1       1

Net input y_inj = [ -2 2 2 2 ]
Output after activation y = [ -1 1 1 1 ]

Result: Correct response is obtained.

Do you want to test another input? (1 for Yes, 0 for No): 1

--------------------------------------------------
Enter the 4 elements of the test vector (space-separated): -1 -1 -1 1

Weight Matrix W:
1       -1      -1      -1
-1      1       1       1
-1      1       1       1
-1      1       1       1

Net input y_inj = [ 0 0 0 0 ]
Output after activation y = [ 0 0 0 0 ]

Result: Incorrect response. The network did not recognize the input.

Do you want to test another input? (1 for Yes, 0 for No): 0
PS C:\Users\prajw\OneDrive\Desktop>
```

```
PS C:\Users\prajw\OneDrive\Desktop> g++ associative.cpp -o associative
PS C:\Users\prajw\OneDrive\Desktop> .\associative

-------------------------------------------------
Enter the 4 elements of the test vector (space-separated): -1 1 1 1

Weight Matrix W:
1       -1      -1      -1
-1      1       1       1
-1      1       1       1
-1      1       1       1

Net input y_inj = [ -4 4 4 4 ]
Output after activation y = [ -1 1 1 1 ]

Result: Correct response is obtained.

Do you want to test another input? (1 for Yes, 0 for No): 1

-------------------------------------------------
Enter the 4 elements of the test vector (space-separated): 0 1 1 1

Weight Matrix W:
1       -1      -1      -1
-1      1       1       1
-1      1       1       1
-1      1       1       1

Net input y_inj = [ -3 3 3 3 ]
Output after activation y = [ -1 1 1 1 ]

Result: Correct response is obtained.

Do you want to test another input? (1 for Yes, 0 for No): 1

-------------------------------------------------
Enter the 4 elements of the test vector (space-separated): -1 -1 1 1

Weight Matrix W:
1       -1      -1      -1
-1      1       1       1
-1      1       1       1
-1      1       1       1
```

CONCLUSION:	In summary, associative memory models—encompassing autoassociative, heteroassociative, and Bidirectional Associative Memory (BAM) networks—serve as efficient computational frameworks for storing and recalling patterns using weight matrices calculated via the outer product rule. These networks demonstrate robust capabilities in retrieving correct responses even when presented with missing or corrupted input data.